

KernelPack RAG — Naive to Hybrid

Pipeline Development: Sections 1–4

May 26, 2026

Overview

Four retrieval pipeline versions were developed on `kernelpack-python`, each introducing one fix targeting a specific observed failure mode. Three fixed benchmark queries ran across all sections to track improvement. Absolute similarity scores differ by section (cosine, L2, RRF) and should not be compared directly — rank position is the consistent signal.

Section	Change	Retrieval	Chunks
1	Baseline — fixed-line chunking	Dense (MiniLM, cosine)	185
2	AST chunking (tree-sitter)	Dense (MiniLM, L2) + ChromaDB	520
3	Code embedding model (UniXcoder)	Dense (UniXcoder, L2)	520
4	MIN_LINES=5 filter + BM25	Hybrid RRF (UniXcoder + BM25)	252

Table 1: Pipeline progression. Each section isolates one change.

Benchmark Query Results (Rank Position)

Q1: *“how to create an RBF-FD differentiation matrix?”*

Q2: *“how to define an ellipse boundary and generate nodes?”*

Q3: *“how to step diffusion forward in time?”*

Query	Rank	S1	S2	S3	S4
Q1	1	<code>_numba.py</code> (wrong)	<code>evaluate_gradient</code> (wrong)	<code>diffusion_solver</code> (getter)	<code>bc.op</code> (adjacent)
	2	<code>poly/core.py</code> (wrong)	<code>phs_rbf</code> (2-line wrapper)	<code>diffusion_solver</code> (getter)	<code>make_assembler</code> (adjacent)
	3	<code>rbffd/core.py</code> (right file)	<code>initialize_geometry</code>	<code>make_assembler</code>	<code>gmres_with_fallback</code> (wrong)
Q2	1	<code>geometry/core.py</code> (right)	<code>generate_interior_nodes</code> ✓	<code>clip_points_by_geometry</code> ✓	<code>clip_points_by_geometry</code> ✓
	2	<code>pu_sl_advection</code> (wrong)	<code>boundary_hit_on_segment</code> (wrong)	<code>build_domain_descriptor</code> ✓	<code>build_domain_descriptor</code> ✓
	3	<code>nodes/core.py</code> (right)	<code>build_planar_parametric_nodes</code>	<code>generate_interior_nodes</code> ✓	<code>validate_bc_config</code> (BM25 regression)
Q3	1	<code>pu_slfd_adv_diff</code> ✓	<code>diffusion_solver</code> (getter)	<code>set_step_size</code> (setter)	<code>set_initial_state</code> (BM25 regression)
	2	<code>pu_slpu_adv_diff</code> ✓	<code>init</code> (wrong function)	<code>set_step_size</code> (setter)	<code>bdf2_step</code> ✓
	3	<code>het_multispecies</code> ✓	<code>diffusion_solver</code> (getter)	<code>forward_sl_step</code> (wrong)	<code>backward_sl_step</code> (wrong)

Table 2: Rank-1–3 results per query per section. ✓ = correct or clearly relevant result. Parenthetical notes indicate the failure mode.

Key Findings

1. **Fixed-line chunking is the primary failure mode.** Splitting at 40-line boundaries cuts across method bodies. No single chunk is self-contained, so no query can reliably retrieve the right function. Q1 and Q2 are both imprecise at baseline despite hitting the right files.
2. **AST chunking makes results attributable but exposes a new problem.** 520 function-level chunks give each result a name and location. Trivial 1–2 line getters and wrappers now pollute the index — the embedding model has almost nothing to work with on a one-liner, but they rank on par with real implementations.
3. **UniXcoder fixes Q2 outright.** Swapping to a code-trained embedding model (Section 3) returns all three correct functions for Q2. Q1 and Q3 remain broken — the model is not the limiting factor there; chunk quality is.
4. **Filtering short functions is necessary but introduces a tradeoff.** `MIN_LINES=5` removes trivial getters (252 chunks, down from 520), which allows real implementations to surface. But BM25 then introduces regressions via keyword over-matching: “state” pulls `set_initial_state` to rank 1 for Q3; “boundary” pulls an irrelevant validator to rank 3 for Q2.
5. **Q1 (`assemble_op`) never appeared across all four sections.** This is a vocabulary gap, not a retrieval failure. The query says “RBF-FD differentiation matrix”; the codebase says `assemble_op`, `FDDiffOp`, `RBFStencil`. Neither dense search nor BM25 can bridge disjoint vocabularies. This is the argument for LLM-augmented indexing.

VDB Recommendation

Feature	ChromaDB	Qdrant
Native hybrid search	dense only	sparse + dense built-in
Persistent storage	yes (disk mode)	yes
Metadata filtering	yes	yes
Native RRF	manual implementation required	built-in
BM25 / sparse vectors	external, in-memory	SparseVector
Scale	small-to-medium	production-grade (HNSW)
Python client	yes	yes

Table 3: ChromaDB vs. Qdrant feature comparison.

ChromaDB was appropriate for this experiment — zero infrastructure overhead, fast iteration, sufficient to surface all four failure modes. Qdrant is the right production choice: the BM25 leg implemented here is not persisted, not calibrated against the dense leg, and not tunable per query. Qdrant’s native sparse + dense support collapses three moving parts into one tool call.

Next Steps

1. **Migrate to Qdrant.** Same 252 filtered chunks, same UniXcoder embeddings, same metadata. Add sparse vectors as a second vector field. Re-run benchmark queries and compare rank positions.

2. **LLM-augmented indexing.** Prepend LLM-generated natural-language summaries to each chunk at index time to close the vocabulary gap that stalled Q1 across all four sections. (Evaluated in notebook 02.)